EPICS Multi-Core Utilities

Generated by Doxygen 1.9.4

Chapter 1

EPICS Multi-Core Utilities

1.1 Scope of this Document

This documentation covers the C API and the iocShell commands of the EPICS Multi-Core Utilities.

1.2 Introduction

The EPICS Multi-Core Utilities library contains tools that allow tweaking of real-time parameters for EPICS IOC threads running on multi-core processors under the Linux operating system.

These tools are intended to set up multi-core IOCs for fast controllers, by:

- Confining either parts or the complete EPICS IOC onto a subset of the available cores, allowing hard real-time applications and threads to run on dedicated cores.
- Changing priorities of callback, driver or communication threads with respect to database processing.
- Selecting real-time scheduling policy (FIFO or Round-Robin) for selected threads.
- Locking the IOC process virtual memory into RAM to avoid swapping.

1.2.1 Advanced Thread Show Routines

An extended version of the `epicsThreadShow()` command, showing scheduling policy and CPU affinity in addition to the usual output.

Details can be found in the documentation for module Real-Time threadShow Routines.

1.2.2 Rule Based Real-Time Property Manipulation

A module allowing to specify rules, which consist of a regular expression to match the thread name against, and a set of commands that allow to specify the real-time properties of a thread.

Whenever the EPICS IOC starts a thread, its name is matched against all existing rules, and for matching rules the commands are applied.

Details can be found in the documentation for module Rule-Based Thread Properties.

Warning

> The default priorities of the EPICS IOC threads are well-chosen. They have been proven to ensure reliable IOC operation and communication, in many installations, under a variety of circumstances.
> Manipulating the real-time properties, especially scheduling policies and priorities, may have unwanted side effects. Use this feature sparingly, and test well.

1.2.3 Memory Locking

A module allowing to lock the IOC process virtual memory into RAM. This makes sure that no swapping occurs, and thus avoids page faults which would introduce latency and lead to indeterministic timing.

Details can be found in the documentation for module Memory Locking.

1.3 Sources

The sources are on GitHub at `https://github.com/epics-modules/MCoreUtils`

They can be checked out using

```
git clone https://github.com/epics-modules/MCoreUtils.git
```

Releases can be found on GitHub (see above) or at `http://sourceforge.net/projects/epics/files/mcoreutil`

1.4 Requirements

- Linux operating system
- `EPICS BASE 3.15` (3.15.1 or later)

1.5 Installation

- Unpack the distribution tar or check out the source tree.
- Run `make`
- To generate a minimal example IOC, run `make -C example`

1.6 Usage

To use the Multi-Core Utilities in an IOC application tree, you have to add a definition to `.../configure/RELEASE` that points to the location of the `mcoreutils` module.

In the directory that builds your IOC binary, the `Makefile` has to make sure the IOC is only built for Linux. Then add the dbd file and the Library, e.g.:

```
...
PROD_IOC_Linux = mcutest
...
mcutest_DBD += mcoreutils.dbd
...
mcutest_LIBS += mcoreutils
...
```

That's it. Enjoy!

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Module Documentation

4.1 Real-Time threadShow Routines

Add two new threadShow functions that show scheduling policy and CPU affinity.

Files

- file threadShow.c

  New threadShow showing real-time properties.

Functions

- epicsShareFunc void mcoreThreadShowInit (void)

  Initialization routine.
- epicsShareFunc void mcoreThreadShow (epicsThreadId thread, unsigned int level)

  iocShell: *Show thread info for one thread.*
- epicsShareFunc void mcoreThreadShowAll (unsigned int level)

  iocShell: *Show thread info for all threads.*

4.1.1 Detailed Description

Add two new threadShow functions that show scheduling policy and CPU affinity.

Adds two new threadShow functions that, in addition to the properties shown by `epicsThreadShow()` and `epicsThreadShowAll()`, print the scheduling policy, and the CPU affinity of each thread.

Uses the `epicsThreadMap()` call to have a hook function being called for every thread, which prints out the thread properties.

4.1.2 Function Documentation

4.1.2.1 mcoreThreadShow()

```
epicsShareFunc void mcoreThreadShow (
            epicsThreadId thread,
            unsigned int level )
```

iocShell: Show thread info for one thread.

Sets the global thread and level variables, and calls the map function.

Parameters

thread	id of thread to show
level	verbosity level

IOC Shell

```
mcoreThreadShow thread level
```

`thread`	thread name or id
`level`	verbosity level

iocShell: Show thread info for one thread.

Definition at line 122 of file threadShow.c.

4.1.2.2 mcoreThreadShowAll()

```
epicsShareFunc void mcoreThreadShowAll (
            unsigned int level )
```

iocShell: Show thread info for all threads.

Parameters

level	verbosity level

IOC Shell

```
mcoreThreadShowAll level
```

`level`	verbosity level

iocShell: Show thread info for all threads.

Definition at line 136 of file threadShow.c.

4.1.2.3 mcoreThreadShowInit()

```
epicsShareFunc void mcoreThreadShowInit (
            void  )
```

Initialization routine.

Must be called before using any of the other functions, which is done when registering the iocsh commands.

Definition at line 154 of file threadShow.c.

4.2 Rule-Based Thread Properties

Allow user-specified rules that modify real-time properties of EPICS threads.

Files

- file threadRules.c

  Rule-based modification of thread real-time properties.

Functions

- epicsShareFunc void mcoreThreadModify (epicsThreadId id, const char ∗policy, const char ∗priority, const char ∗cpus)

  iocShell: *Modify a thread's real-time properties.*
- epicsShareFunc void mcoreThreadRulesInit ()

  Initialization routine.
- epicsShareFunc long mcoreThreadRuleAdd (const char ∗name, const char ∗policy, const char ∗priority, const char ∗cpus, const char ∗pattern)

  iocShell: *Add or replace a thread rule.*
- epicsShareFunc void mcoreThreadRuleDelete (const char ∗name)

  iocShell: *Delete a thread rule.*
- epicsShareFunc void mcoreThreadRulesShow (void)

  iocShell: *Print a comprehensive list of the thread rules.*

4.2.1 Detailed Description

Allow user-specified rules that modify real-time properties of EPICS threads.

Implements a library that uses rules to modify real-time properties of EPICS threads:

- *Scheduling policy*
  Scheduling mechanism used for this thread. When POSIX scheduling is enabled, the default mechanism is `SCHED_FIFO`, but `SCHED_OTHER` and `SCHED_RR` are also supported.
- *Scheduling priority*
  OSI priority value that gets converted to the system's real-time priority schema.
- *CPU Affinity*
  Set of CPUs that this thread is allowed to run on.

This is achieved by creating a linked list of rules, which consist of a regular expression pattern and modification instructions. A hook function is added to the EPICS thread creation module. The hook is called from every thread as part of its creation, matches the regular expression patterns of all rules against the name of the newly created thread, and applies the modifications of all rules that match.

See man pages for `pthread_setschedparam(3)` and `sched_setscheduler(2)` for details on scheduling policy and priority, `pthread_setaffinity_np(3)` and `sched_setaffinity(2)` for details on CPU affinity.

Configuration Files

The module tries to read a system configuration file (`/etc/rtrules`) and a user configuration file (default: `$HOME/.rtrules`) to create the initial list of thread rules.

The file format is based on the format of the `/etc/rtgroups` file on RHEL-MRG. Each line has the format

`name:policy:priority:affinity:pattern`

`name`	name of the rule
`policy`	scheduling policy to set for the thread (first letter, not case sensitive), * = don't change
`priority`	scheduling priority to set for the thread (a + or − sign adds to the current priority), * = don't change
`affinity`	CPUs to set the thread's affinity to (use , and − to specify multiple CPUs and ranges, e.g. `0`,3-5), * = don't change
`pattern`	regular expression pattern to match thread names against, see man page for `regex(7)` for details

Lines starting with # (comments), and empty lines (containing only whitespace) are ignored.

Environment Variables

`HOME` location of the `HOME` directory (default: `/`)

`EPICS_MCORE_USERCONFIG` name of user configuration file, relative to the `HOME` directory (default: `.rtrules`)

Linux Security

To change its scheduling policy and priority, under modern Linux systems the process must have an `rtprio` entry in the pam limits module configuration.

See the `limits.conf(5)` man page for details.

Known Issues

A thread calling `epicsThreadSetPriority()` to set its priority while running may override the priorities defined in the rules at any time.

4.2.2 Function Documentation

4.2.2.1 mcoreThreadModify()

```
epicsShareFunc void mcoreThreadModify (
            epicsThreadId id,
            const char * policy,
            const char * priority,
            const char * cpus )
```

iocShell: Modify a thread's real-time properties.

Parameters

id	EPICS thread id
policy	scheduling policy to set (∗ = don't change)
priority	scheduling priority (OSI) to set (a + or − sign adds to the current priority, ∗ = don't change)
cpus	cpuset specification to set (use , and − to specify multiple CPUs and ranges, ∗ = don't change)

IOC Shell

`mcoreThreadModify thread policy priority cpus`

`thread`	thread name or id
`policy`	scheduling policy to set (∗ = don't change)
`priority`	scheduling priority (OSI) to set (a + or − sign adds to the current priority, ∗ = don't change)
`cpus`	cpuset specification to set (use , and − to specify multiple CPUs and ranges, ∗ = don't change)

iocShell: Modify a thread's real-time properties.

Definition at line 291 of file threadRules.c.

4.2.2.2 mcoreThreadRuleAdd()

```
epicsShareFunc long mcoreThreadRuleAdd (
            const char * name,
            const char * policy,
            const char * priority,
            const char * cpus,
            const char * pattern )
```

iocShell: Add or replace a thread rule.

Parameters

name	rule name (identifier)

Parameters

policy	scheduling policy to set (∗ = don't change)
priority	scheduling priority (OSI) to set (a + or − sign adds to the current priority, ∗ = don't change)
cpus	cpuset specification to set (use , and − to specify multiple CPUs and ranges, ∗ = don't change)
pattern	`regex(7)` pattern to match thread names against

Returns

(OK, ERROR) as (0,-1)

IOC Shell

`mcoreThreadRuleAdd name policy priority cpus pattern`

`name`	rule name (identifier)
`policy`	scheduling policy to set (∗ = don't change)
`priority`	scheduling priority (OSI) to set (a + or − sign adds to the current priority, ∗ = don't change)
`cpus`	cpuset specification to set (use , and − to specify multiple CPUs and ranges, ∗ = don't change)
`pattern`	`regex(7)` pattern to match thread names against

iocShell: Add or replace a thread rule.

Definition at line 109 of file threadRules.c.

4.2.2.3 mcoreThreadRuleDelete()

```
epicsShareFunc void mcoreThreadRuleDelete (
            const char * name )
```

iocShell: Delete a thread rule.

Parameters

name	name (identifier) of the rule to delete

IOC Shell

`mcoreThreadRuleDelete name`

`name`	name (identifier) of the rule to delete

iocShell: Delete a thread rule.

Definition at line 142 of file threadRules.c.

4.2.2.4 mcoreThreadRulesInit()

```
epicsShareFunc void mcoreThreadRulesInit ( )
```

Initialization routine.

Must be called before using any of the other functions, which is done when registering the iocsh commands.

Definition at line 379 of file threadRules.c.

4.2.2.5 mcoreThreadRulesShow()

```
epicsShareFunc void mcoreThreadRulesShow (
            void  )
```

iocShell: Print a comprehensive list of the thread rules.

Rule names are shortened to 16 characters.

IOC Shell

```
mcoreThreadRulesShow
```

iocShell: Print a comprehensive list of the thread rules.

Definition at line 167 of file threadRules.c.

4.3 Memory Locking

Add functions for locking the process memory into RAM.

Files

- file memLock.c

  Locking process memory into RAM.

Functions

- epicsShareFunc void mcoreMLock (void)

  iocShell: *Lock all process virtual memory into RAM.*
- epicsShareFunc void mcoreMUnlock (void)

  iocShell: *Unlock process virtual memory from RAM.*

4.3.1 Detailed Description

Add functions for locking the process memory into RAM.

Adds functions that allow locking and unlocking the process virtual memory into RAM to make sure no page faults occur, which would introduce unpredictable interruptions and latency.

See man page for `mlockall(2)` for more details on memory locking.

Linux Security (non-systemd)

To allow locking all its memory, under modern Linux systems the process must have a `memlock` entry in the pam limits module configuration.

See the `limits.conf(5)` man page for details.

Linux Security (systemd)

If an IOC process is started by systemd the following configuration is required in the unit file to grant the IOC process permission to adjust real-time priorities as well as to lock memory:

```
[Service]
LimitMEMLOCK=infinity
LimitRTPRIO=99
```

Refer to the `systemd documentation` for details.

Use with EPICS >=3.15.4

Starting with EPICS Base 3.15.4, IOCs are automatically calling `mlockall` if the IOC process is allowed to set different thread priorities (that is if `sched_get_priority_max() > sched_get_priority_min()`). The following commands are thus only needed with older IOCs as well as for troubleshooting.

4.3.2 Function Documentation

4.3.2.1 mcoreMLock()

```
epicsShareFunc void mcoreMLock (
            void  )
```

iocShell: Lock all process virtual memory into RAM.

IOC Shell

`mcoreMLock`

Definition at line 34 of file memLock.c.

4.3.2.2 mcoreMUnlock()

```
epicsShareFunc void mcoreMUnlock (
            void  )
```

iocShell: Unlock process virtual memory from RAM.

IOC Shell

`mcoreMUnlock`

Definition at line 40 of file memLock.c.

Chapter 5

File Documentation

5.1 mcoreutils.h File Reference

```
#include <unistd.h>
#include <epicsThread.h>
#include <shareLib.h>
```

Functions

- epicsShareFunc void mcoreThreadShowInit (void)

  Initialization routine.
- epicsShareFunc void mcoreThreadShow (epicsThreadId thread, unsigned int level)

  iocShell: *Show thread info for one thread.*
- epicsShareFunc void mcoreThreadShowAll (unsigned int level)

  iocShell: *Show thread info for all threads.*
- epicsShareFunc void mcoreThreadModify (epicsThreadId id, const char ∗policy, const char ∗priority, const char ∗cpus)

  iocShell: *Modify a thread's real-time properties.*
- epicsShareFunc void mcoreThreadRulesInit ()

  Initialization routine.
- epicsShareFunc long mcoreThreadRuleAdd (const char ∗name, const char ∗policy, const char ∗priority, const char ∗cpus, const char ∗pattern)

  iocShell: *Add or replace a thread rule.*
- epicsShareFunc void mcoreThreadRuleDelete (const char ∗name)

  iocShell: *Delete a thread rule.*
- epicsShareFunc void mcoreThreadRulesShow (void)

  iocShell: *Print a comprehensive list of the thread rules.*
- epicsShareFunc void mcoreMLock (void)

  iocShell: *Lock all process virtual memory into RAM.*
- epicsShareFunc void mcoreMUnlock (void)

  iocShell: *Unlock process virtual memory from RAM.*

5.1.1 Detailed Description

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file mcoreutils.h.

5.2 mcoreutils.h

Go to the documentation of this file.

```
00001 /*************************************************/
00107 #ifndef MCOREUTILS_H
00108 #define MCOREUTILS_H
00109
00110 #include <unistd.h>
00111
00112 #include <epicsThread.h>
00113 #include <shareLib.h>
00114
00115 #ifdef __cplusplus
00116 extern "C" {
00117 #endif
00118
00138 epicsShareFunc void mcoreThreadShowInit(void);
00139
00154 epicsShareFunc void mcoreThreadShow(epicsThreadId thread, unsigned int level);
00155
00167 epicsShareFunc void mcoreThreadShowAll(unsigned int level);
00168
00270 epicsShareFunc void mcoreThreadModify(epicsThreadId id,
00271                                       const char *policy,
00272                                       const char *priority,
00273                                       const char *cpus);
00274
00281 epicsShareFunc void mcoreThreadRulesInit();
00282
00310 epicsShareFunc long mcoreThreadRuleAdd(const char *name,
00311                                        const char *policy,
00312                                        const char *priority,
00313                                        const char *cpus,
00314                                        const char *pattern);
00315
00327 epicsShareFunc void mcoreThreadRuleDelete(const char *name);
00328
00337 epicsShareFunc void mcoreThreadRulesShow(void);
00338
00388 epicsShareFunc void mcoreMLock(void);
00389
00396 epicsShareFunc void mcoreMUnlock(void);
00397
00402 #ifdef __cplusplus
00403 }
00404 #endif
00405
00406 #endif // MCOREUTILS_H
```

5.3 memLock.c File Reference

Locking process memory into RAM.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <sys/mman.h>
#include <errlog.h>
#include <shareLib.h>
#include "mcoreutils.h"
```

Functions

- void mcoreMLock (void)

  iocShell: *Lock all process virtual memory into RAM.*
- void mcoreMUnlock (void)

  iocShell: *Unlock process virtual memory from RAM.*

5.3.1 Detailed Description

Locking process memory into RAM.

Author

Ralph Lange Ralph.Lange@gmx.de

Dirk Zimoch Dirk.Zimoch@psi.ch

Copyright

Definition in file memLock.c.

5.4 memLock.c

Go to the documentation of this file.

```
00001 /*************************************************/
00021 #include <stdio.h>
00022 #include <string.h>
00023 #include <errno.h>
00024 #include <sys/mman.h>
00025
00026 #include <errlog.h>
00027 #include <shareLib.h>
00028
00030 #define epicsExportSharedSymbols
00032 #include "mcoreutils.h"
00033
00034 void mcoreMLock(void) {
00035     if (mlockall(MCL_CURRENT|MCL_FUTURE)) {
00036         errlogPrintf("mlockall error %s\n", strerror(errno));
00037     }
00038 }
00039
00040 void mcoreMUnlock(void) {
00041     if (munlockall()) {
00042         errlogPrintf("munlockall error %s\n", strerror(errno));
00043     }
00044 }
00045
```

5.5 shellCommands.c File Reference

iocShell registration of MCoreUtils commands.

```
#include <unistd.h>
#include <stdlib.h>
#include <iocsh.h>
#include <epicsExport.h>
#include <epicsThread.h>
#include "mcoreutils.h"
```

5.5.1 Detailed Description

iocShell registration of MCoreUtils commands.

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file shellCommands.c.

5.6 shellCommands.c

Go to the documentation of this file.

```
00001 /*************************************************/
00012 #include <unistd.h>
00013 #include <stdlib.h>
00014
00015 #include <iocsh.h>
00016 #include <epicsExport.h>
00017 #include <epicsThread.h>
00018
00019 #include "mcoreutils.h"
00020
00026 static epicsThreadId getThreadIdFor(const char* thread) {
00027     const char *cp;
00028     epicsThreadId tid;
00029     unsigned long ltmp;
00030     char *endp;
00031
00032     if (!thread) return 0;
00033     cp = thread;
00034     ltmp = strtoul(cp, &endp, 0);
00035     if (*endp) {
00036         tid = epicsThreadGetId(cp);
00037         if (!tid) {
00038             printf("*** %s is not a valid thread name ***\n", cp);
00039         }
00040     }
00041     else {
00042         tid = (epicsThreadId) ltmp;
00043     }
00044     return tid;
00045 }
```

```
00046
00047 static const iocshArg mcoreThreadShowArg0 = {"thread", iocshArgString};
00048 static const iocshArg mcoreThreadShowArg1 = {"level", iocshArgInt};
00049 static const iocshArg *const mcoreThreadShowArgs[] = {
00050     &mcoreThreadShowArg0,
00051     &mcoreThreadShowArg1,
00052 };
00053 static const iocshFuncDef mcoreThreadShowDef =
00054     {"mcoreThreadShow", 2, mcoreThreadShowArgs};
00055 static void mcoreThreadShowCall(const iocshArgBuf * args) {
00056     epicsThreadId tid;
00057     unsigned int level = args[1].ival;
00058     if (!args[0].sval) {
00059         printf("Missing argument\nUsage:  mcoreThreadShow thread [level]\n");
00060         return;
00061     }
00062     tid = getThreadIdFor(args[0].sval);
00063     if (tid) {
00064         mcoreThreadShow(  0, level);
00065         mcoreThreadShow(tid, level);
00066     }
00067 }
00068
00069 static const iocshArg mcoreThreadShowAllArg0 = {"level", iocshArgInt};
00070 static const iocshArg *const mcoreThreadShowAllArgs[] = {
00071     &mcoreThreadShowAllArg0,
00072 };
00073 static const iocshFuncDef mcoreThreadShowAllDef =
00074     {"mcoreThreadShowAll", 1, mcoreThreadShowAllArgs};
00075 static void mcoreThreadShowAllCall(const iocshArgBuf * args) {
00076     unsigned int level = args[0].ival;
00077     mcoreThreadShowAll(level);
00078 }
00079
00080 static const iocshArg mcoreThreadRuleAddArg0 = {"name", iocshArgString};
00081 static const iocshArg mcoreThreadRuleAddArg1 = {"policy", iocshArgString};
00082 static const iocshArg mcoreThreadRuleAddArg2 = {"priority", iocshArgString};
00083 static const iocshArg mcoreThreadRuleAddArg3 = {"cpuset", iocshArgString};
00084 static const iocshArg mcoreThreadRuleAddArg4 = {"pattern", iocshArgString};
00085 static const iocshArg *const mcoreThreadRuleAddArgs[] = {
00086     &mcoreThreadRuleAddArg0,
00087     &mcoreThreadRuleAddArg1,
00088     &mcoreThreadRuleAddArg2,
00089     &mcoreThreadRuleAddArg3,
00090     &mcoreThreadRuleAddArg4,
00091 };
00092 static const iocshFuncDef mcoreThreadRuleAddDef =
00093     {"mcoreThreadRuleAdd", 5, mcoreThreadRuleAddArgs};
00094 static void mcoreThreadRuleAddCall(const iocshArgBuf * args) {
00095     int i;
00096     char missing = 0;
00097     for (i=0; i<5; i++) {
00098         if (NULL == args[i].sval) missing |= 1;
00099     }
00100     if (missing) {
00101         printf("Missing argument\nUsage:  mcoreThreadRuleAdd name policy priority cpuset pattern\n");
00102         return;
00103     }
00104     mcoreThreadRuleAdd(args[0].sval, args[1].sval, args[2].sval, args[3].sval, args[4].sval);
00105 }
00106
00107 static const iocshArg mcoreThreadRuleDeleteArg0 = {"name", iocshArgString};
00108 static const iocshArg *const mcoreThreadRuleDeleteArgs[] = {
00109     &mcoreThreadRuleDeleteArg0,
00110 };
00111 static const iocshFuncDef mcoreThreadRuleDeleteDef =
00112     {"mcoreThreadRuleDelete", 1, mcoreThreadRuleDeleteArgs};
00113 static void mcoreThreadRuleDeleteCall(const iocshArgBuf * args) {
00114     if (NULL == args[0].sval) {
00115         printf("Missing argument\nUsage:  mcoreThreadRuleDelete name\n");
00116         return;
00117     }
00118     mcoreThreadRuleDelete(args[0].sval);
00119 }
00120
00121 static const iocshFuncDef mcoreThreadRulesShowDef =
00122     {"mcoreThreadRulesShow", 0, NULL};
00123 static void mcoreThreadRulesShowCall(const iocshArgBuf * args) {
00124     mcoreThreadRulesShow();
00125 }
00126
00127 static const iocshArg mcoreThreadModifyArg0 = {"thread", iocshArgString};
00128 static const iocshArg mcoreThreadModifyArg1 = {"policy", iocshArgString};
00129 static const iocshArg mcoreThreadModifyArg2 = {"priority", iocshArgString};
00130 static const iocshArg mcoreThreadModifyArg3 = {"cpuset", iocshArgString};
00131 static const iocshArg *const mcoreThreadModifyArgs[] = {
00132     &mcoreThreadModifyArg0,
```

```
00133     &mcoreThreadModifyArg1,
00134     &mcoreThreadModifyArg2,
00135     &mcoreThreadModifyArg3,
00136 };
00137 static const iocshFuncDef mcoreThreadModifyDef =
00138     {"mcoreThreadModify", 4, mcoreThreadModifyArgs};
00139 static void mcoreThreadModifyCall(const iocshArgBuf * args) {
00140     epicsThreadId tid;
00141     int i;
00142     char missing = 0;
00143
00144     for (i = 0; i < 4; i++) {
00145         if (NULL == args[i].sval) missing |= 1;
00146     }
00147     if (missing) {
00148         printf("Missing argument\nUsage:  mcoreThreadModify thread  policy  priority  cpuset\n");
00149         return;
00150     }
00151     tid = getThreadIdFor(args[0].sval);
00152     if (tid)
00153         mcoreThreadModify(tid, args[1].sval, args[2].sval, args[3].sval);
00154 }
00155
00156 static const iocshFuncDef mcoreMLockDef =
00157     {"mcoreMLock", 0, NULL};
00158 static void mcoreMLockCall(const iocshArgBuf * args) {
00159     mcoreMLock();
00160 }
00161
00162 static const iocshFuncDef mcoreMUnlockDef =
00163     {"mcoreMUnlock", 0, NULL};
00164 static void mcoreMUnlockCall(const iocshArgBuf * args) {
00165     mcoreMUnlock();
00166 }
00167
00168 static void mcoreRegister(void)
00169 {
00170     static int firstTime = 1;
00171     if(!firstTime) return;
00172     firstTime = 0;
00173
00174     mcoreThreadShowInit();
00175     mcoreThreadRulesInit();
00176     iocshRegister(&mcoreThreadShowDef,       mcoreThreadShowCall);
00177     iocshRegister(&mcoreThreadShowAllDef,    mcoreThreadShowAllCall);
00178     iocshRegister(&mcoreThreadRuleAddDef,    mcoreThreadRuleAddCall);
00179     iocshRegister(&mcoreThreadRuleDeleteDef, mcoreThreadRuleDeleteCall);
00180     iocshRegister(&mcoreThreadRulesShowDef,  mcoreThreadRulesShowCall);
00181     iocshRegister(&mcoreThreadModifyDef,     mcoreThreadModifyCall);
00182     iocshRegister(&mcoreMLockDef,            mcoreMLockCall);
00183     iocshRegister(&mcoreMUnlockDef,          mcoreMUnlockCall);
00184 }
00186 epicsExportRegistrar(mcoreRegister);
```

5.7 threadRules.c File Reference

Rule-based modification of thread real-time properties.

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <sys/types.h>
#include <regex.h>
#include <string.h>
#include <ellLib.h>
#include <envDefs.h>
#include <errlog.h>
#include <epicsStdio.h>
#include <epicsMath.h>
#include <epicsThread.h>
#include <epicsMutex.h>
#include <shareLib.h>
#include "utils.h"
```

```
#include "mcoreutils.h"
```

- typedef struct threadRule threadRule

  A thread rule.
- long mcoreThreadRuleAdd (const char *name, const char *policy, const char *priority, const char *cpus, const char *pattern)

  Add or replace a thread rule.
- void mcoreThreadRuleDelete (const char *name)

  Delete a thread rule.
- void mcoreThreadRulesShow (void)

  Print a comprehensive list of the thread rules.
- void mcoreThreadModify (epicsThreadId id, const char *policy, const char *priority, const char *cpus)

  Modify a thread's real-time properties.
- void mcoreThreadRulesInit (void)

  Initialization routine.

5.7.1 Detailed Description

Rule-based modification of thread real-time properties.

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file threadRules.c.

5.7.2 Typedef Documentation

5.7.2.1 threadRule

```
typedef struct threadRule threadRule
```

A thread rule.

Used to manipulate real-time properties when threads are started. The thread rules are kept in a linked list.

5.8 threadRules.c

Go to the documentation of this file.

```
00001 /*************************************************/
00019 #include <stdlib.h>
00020 #include <stdio.h>
00021 #include <pthread.h>
00022 #include <sys/types.h>
00023 #include <regex.h>
00024 #include <string.h>
00025
00026 #include <ellLib.h>
00027 #include <envDefs.h>
00028 #include <errlog.h>
00029 #include <epicsStdio.h>
00030 #include <epicsMath.h>
00031 #include <epicsThread.h>
00032 #include <epicsMutex.h>
00033 #include <shareLib.h>
00034
00035 #include "utils.h"
00036
00038 #define epicsExportSharedSymbols
00040 #include "mcoreutils.h"
00041
00043 extern int epicsThreadGetPosixPriority(epicsThreadOSD *pthreadInfo);
00045
00052 typedef struct threadRule {
00053     ELLNODE     node;
00054     char       *name;
00055     char       *pattern;
00056     char       *cpus;
00057     regex_t     reg;
00058     char        ch_policy;
00059     char        ch_priority;
00060     char        ch_affinity;
00061     char        rel_priority;
00062     int         policy;
00063     int         priority;
00064     cpu_set_t   cpuset;
00065 } threadRule;
00066
00067 static ELLLIST threadRules = ELLLIST_INIT;
00068 static epicsMutexId listLock;
00069 static unsigned int cpuspecLen;
00070 static char *sysConfigFile      = "/etc/rtrules";
00071 static ENV_PARAM userHome       = {"HOME","/"};
00072 static ENV_PARAM userConfigFile = {"EPICS_MCORE_USERCONFIG",".rtrules"};
00073
00082 static void parseModifiers(threadRule *prule, const char *policy, const char *priority, const char
      *cpus)
00083 {
00084     if (policy && '*' != policy[0] && '\0' != policy[0]) {
00085         prule->ch_policy = 1;
00086         prule->policy = strToPolicy(policy);
00087         if (-1 == prule->policy) {
00088             prule->ch_policy = prule->policy = 0;
00089         }
00090     }
00091     if (priority && '*' != priority[0] && '\0' != priority[0]) {
00092         prule->ch_priority = 1;
00093         if ('+' == priority[0] || '-' == priority[0]) {
00094             prule->rel_priority = 1;
00095         }
00096         prule->priority = atoi(priority);
00097         if (prule->priority > epicsThreadPriorityMax) prule->priority = epicsThreadPriorityMax;
00098         if (prule->priority < epicsThreadPriorityMin) prule->priority = epicsThreadPriorityMin;
00099     }
00100     if (cpus && '*' != cpus[0] && '\0' != cpus[0]) {
00101         prule->ch_affinity = 1;
00102         strToCpuset(&prule->cpuset, cpus);
00103     }
00104 }
00105
00109 long mcoreThreadRuleAdd(const char *name, const char *policy, const char *priority, const char *cpus,
      const char *pattern)
00110 {
00111     threadRule *prule;
00112
00113     prule = calloc(1,sizeof(threadRule));
00114     if (!prule) {
00115         errlogPrintf("Memory allocation error\n");
00116         return -1;
00117     }
00118
```

```
00119     prule->name    = strdup(name);
00120     prule->pattern = strdup(pattern);
00121     prule->cpus    = strdup(cpus);
00122     if (!prule->name || !prule->pattern || !prule->cpus) {
00123         errlogPrintf("Memory allocation error\n");
00124         free(prule->name); free(prule->pattern); free(prule->cpus);
00125         free(prule);
00126         return -1;
00127     }
00128
00129     parseModifiers(prule, policy, priority, cpus);
00130     regcomp(&prule->reg, prule->pattern, (REG_EXTENDED || REG_NOSUB));
00131
00132     epicsMutexLock(listLock);
00133     mcoreThreadRuleDelete(name);
00134     ellAdd(&threadRules, &prule->node);
00135     epicsMutexUnlock(listLock);
00136     return 0;
00137 }
00138
00142 void mcoreThreadRuleDelete(const char *name)
00143 {
00144     threadRule *prule;
00145
00146     epicsMutexLock(listLock);
00147     prule = (threadRule *) ellFirst(&threadRules);
00148     while (prule) {
00149         if (0 == strcmp(name, prule->name)) {
00150             ellDelete(&threadRules, &prule->node);
00151             epicsMutexUnlock(listLock);
00152             free(prule->name);
00153             free(prule->pattern);
00154             free(prule->cpus);
00155             regfree(&prule->reg);
00156             free(prule);
00157             return;
00158         }
00159         prule = (threadRule *) ellNext(&prule->node);
00160     }
00161     epicsMutexUnlock(listLock);
00162 }
00163
00167 void mcoreThreadRulesShow(void)
00168 {
00169     threadRule *prule;
00170     const int buflen = 128; //FIXME should be ~ NO_OF_CPUS
00171     char buf[buflen];
00172
00173     epicsMutexLock(listLock);
00174     prule = (threadRule *) ellFirst(&threadRules);
00175     if (!prule) {
00176         fprintf(epicsGetStdout(), "No rules defined.\n");
00177         epicsMutexUnlock(listLock);
00178         return;
00179     }
00180     fprintf(epicsGetStdout(), "            NAME  PRIO POLICY %-*s PATTERN\n", cpuspecLen, "AFFINITY");
00181     while (prule) {
00182         cpusetToStr(buf, buflen, &prule->cpuset);
00183         fprintf(epicsGetStdout(), "%16s  ",
00184                 prule->name);
00185         if (prule->ch_priority) {
00186             if (prule->rel_priority) {
00187                 fprintf(epicsGetStdout(), "%+4d ", prule->priority);
00188             } else {
00189                 fprintf(epicsGetStdout(), "%4d ", prule->priority);
00190             }
00191         } else {
00192             fprintf(epicsGetStdout(), "    * ");
00193         }
00194         fprintf(epicsGetStdout(),"%6s %-*s %s\n",
00195                 prule->ch_policy?policyToStr(prule->policy):"*",
00196                 cpuspecLen, prule->ch_affinity?buf:"*",
00197                 prule->pattern
00198                 );
00199         prule = (threadRule *) ellNext(&prule->node);
00200     }
00201     epicsMutexUnlock(listLock);
00202 }
00203
00210 static void modifyRTProperties(epicsThreadId id, threadRule *prule)
00211 {
00212     int status;
00213
00214     if (prule->ch_policy || prule->ch_priority) {
00215         status = pthread_attr_getschedparam(&id->attr, &id->schedParam);
00216         if (errVerbose)
00217             checkStatus(status,"pthread_attr_getschedparam");
```

```
00218         status = pthread_attr_getschedpolicy(&id->attr, &id->schedPolicy);
00219         if (errVerbose)
00220             checkStatus(status,"pthread_attr_getschedpolicy");
00221
00222         if (prule->ch_policy) {
00223             id->schedPolicy = prule->policy;
00224             status = pthread_attr_setschedpolicy(&id->attr, id->schedPolicy);
00225             if (errVerbose)
00226                 checkStatus(status,"pthread_attr_setschedpolicy");
00227             if (SCHED_FIFO == prule->policy || SCHED_RR == prule->policy) {
00228                 id->isRealTimeScheduled = 1;
00229             } else {
00230                 id->isRealTimeScheduled = 0;
00231             }
00232         }
00233
00234         if (prule->ch_priority) {
00235             unsigned int priority;
00236             if (prule->rel_priority) {
00237                 priority = id->osiPriority + prule->priority;
00238                 if (priority > epicsThreadPriorityMax) priority = epicsThreadPriorityMax;
00239                 if (priority < epicsThreadPriorityMin) priority = epicsThreadPriorityMin;
00240             } else {
00241                 priority = prule->priority;
00242             }
00243             id->osiPriority = priority;
00244             id->schedParam.sched_priority = epicsThreadGetPosixPriority(id);
00245             status = pthread_attr_setschedparam(&id->attr, &id->schedParam);
00246             if (errVerbose)
00247                 checkStatus(status,"pthread_attr_setschedparam");
00248         }
00249
00250         status = pthread_setschedparam(id->tid, id->schedPolicy, &id->schedParam);
00251         if (errVerbose)
00252             checkStatus(status,"pthread_setschedparam");
00253     }
00254
00255     if (prule->ch_affinity) {
00256         status = pthread_attr_setaffinity_np(&id->attr,
00257                                              sizeof(cpu_set_t),
00258                                              &prule->cpuset);
00259         if (errVerbose)
00260             checkStatus(status,"pthread_attr_setaffinity_np");
00261         status = pthread_setaffinity_np(id->tid,
00262                                         sizeof(cpu_set_t),
00263                                         &prule->cpuset);
00264         if (errVerbose)
00265             checkStatus(status,"pthread_setaffinity_np");
00266     }
00267 }
00268
00269 static void threadStartHook (epicsThreadId id)
00270 {
00271     threadRule *prule;
00272
00273     epicsMutexLock(listLock);
00274     prule = (threadRule *) ellFirst(&threadRules);
00275     if (!prule) {
00276         epicsMutexUnlock(listLock);
00277         return;
00278     }
00279     while (prule) {
00280         if (0 == regexec(&prule->reg, id->name, 0, NULL, 0)) {
00281             modifyRTProperties(id, prule);
00282         }
00283         prule = (threadRule *) ellNext(&prule->node);
00284     }
00285     epicsMutexUnlock(listLock);
00286 }
00287
00291 void mcoreThreadModify(epicsThreadId id, const char *policy, const char *priority, const char *cpus)
00292 {
00293     threadRule rule;
00294
00295     assert(id);
00296     parseModifiers(&rule, policy, priority, cpus);
00297     modifyRTProperties(id, &rule);
00298 }
00299
00306 static int readRulesFromFile(const char *file)
00307 {
00308     const int linelen = 256;
00309     const char sep = ':';
00310     char line[linelen];
00311     int count = 0;
00312
00313     FILE *fp = fopen(file, "r");
```

```
00314     if (NULL == fp) {
00315         if (errVerbose)
00316             errlogPrintf("mcoreThreadRules:  can't open rules file %s\n", file);
00317     } else {
00318         unsigned int lineno = 0;
00319         char *args[5];           // rtgroups format -- name:policy:priority:affinity:pattern
00320         while (fgets(line, linelen, fp)) {
00321             int i;
00322             char *sp;
00323             char *cp;
00324             lineno++;
00325             args[0] = cp = line;
00326             cp += strspn(cp, " \t\r\n");   // trim leading whitespace and empty lines
00327             if (*cp == '#' || *cp == '\0')
00328                 continue;
00329             for (i = 1; i < 5; i++) {
00330                 sp = strchr(cp, sep);
00331                 if (!sp) {
00332                     errlogPrintf("mcoreThreadRules:  error parsing line %d of file %s\n", lineno,
      file);
00333                     return count;
00334                 }
00335                 *sp++ = '\0';
00336                 args[i] = cp = sp;
00337             }
00338             if ((sp = strpbrk(cp, "\n\r"))) {
00339                 *sp = '\0';
00340             }
00341             mcoreThreadRuleAdd(args[0], args[1], args[2], args[3], args[4]);
00342             count++;
00343         }
00344         fclose (fp);
00345     }
00346     return count;
00347 }
00348
00349 static void once(void *arg)
00350 {
00351     const int len = 256;
00352     char userFile[len];
00353     char userRel[len];
00354     int count;
00355
00356     cpuspecLen = (int) (log10(NO_OF_CPUS-1) + 2) * NO_OF_CPUS / 2;
00357     if (cpuspecLen < 10)
00358         cpuspecLen = 10;
00359     listLock = epicsMutexMustCreate();
00360
00361     envGetConfigParam(&userHome, sizeof(userFile), userFile);
00362     envGetConfigParam(&userConfigFile, sizeof(userRel), userRel);
00363     if (userFile[strlen(userFile)-1] != '/')
00364         strcat(userFile, "/");
00365     strncat(userFile, userRel, len-strlen(userFile)-1);
00366
00367     count = readRulesFromFile(sysConfigFile);
00368     printf("MCoreUtils:  Read %d thread rule(s) from %s\n", count, sysConfigFile);
00369
00370     count = readRulesFromFile(userFile);
00371     printf("MCoreUtils:  Read %d thread rule(s) from %s\n", count, userFile);
00372
00373     epicsThreadHookAdd(threadStartHook);
00374 }
00375
00379 void mcoreThreadRulesInit(void)
00380 {
00381     static epicsThreadOnceId onceFlag = EPICS_THREAD_ONCE_INIT;
00382     epicsThreadOnce(&onceFlag, once, NULL);
00383 }
00384
```

5.9 threadShow.c File Reference

New threadShow showing real-time properties.

```
#include <stdlib.h>
#include <sched.h>
#include <string.h>
#include <pthread.h>
```

```
#include <ellLib.h>
#include <errlog.h>
#include <epicsStdio.h>
#include <epicsEvent.h>
#include <epicsThread.h>
#include <epicsMath.h>
#include <shareLib.h>
#include "utils.h"
#include "mcoreutils.h"
```

- void mcoreThreadShow (epicsThreadId thread, unsigned int level)

  Show thread info for one thread.
- void mcoreThreadShowAll (unsigned int level)

  Show thread info for all threads.
- void mcoreThreadShowInit (void)

  Initialization routine.

5.9.1 Detailed Description

New threadShow showing real-time properties.

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file threadShow.c.

5.10 threadShow.c

Go to the documentation of this file.

```
00001 /*************************************************/
00019 #include <stdlib.h>
00020 #include <sched.h>
00021 #include <string.h>
00022 #include <pthread.h>
00023
00024 #include <ellLib.h>
00025 #include <errlog.h>
00026 #include <epicsStdio.h>
00027 #include <epicsEvent.h>
00028 #include <epicsThread.h>
00029 #include <epicsMath.h>
00030 #include <shareLib.h>
00031
00032 #include "utils.h"
00033
00035 #define epicsExportSharedSymbols
00037 #include "mcoreutils.h"
00038
00039 static epicsThreadId showThread;
```

```
00040 static unsigned int showLevel;
00041 static char *buffer;
00042 static char *cpuspec;
00043 static const char *policies;
00044
00051 static void mcoreThreadShowPrint(epicsThreadOSD *pthreadInfo, unsigned int level)
00052 {
00053     if (!pthreadInfo) {
00054         fprintf(epicsGetStdout(), "            NAME       EPICS ID    "
00055             "LWP ID   OSIPRI  OSSPRI  STATE  POLICY CPUSET\n");
00056     } else {
00057         struct sched_param param;
00058         int priority = 0;
00059         int policy;
00060
00061         policies = "?";
00062         cpuspec[0] = '?'; cpuspec[1] = '\0';
00063         if (pthreadInfo->tid) {
00064             cpu_set_t cpuset;
00065             int status;
00066             status = pthread_getschedparam(pthreadInfo->tid,
00067                                            &policy,
00068                                            &param);
00069             if (errVerbose)
00070                 checkStatus(status,"pthread_getschedparam");
00071
00072             if (!status) {
00073                 priority = param.sched_priority;
00074                 policies = policyToStr(policy);
00075             }
00076
00077             status = pthread_getaffinity_np(pthreadInfo->tid,
00078                                             sizeof(cpu_set_t),
00079                                             &cpuset);
00080             if (!status) {
00081                 cpusetToStr(cpuspec, NO_OF_CPUS+2, &cpuset);
00082             }
00083         }
00084
00085         fprintf(epicsGetStdout(),"%16.16s %14p %8lu    %3d%8d %8.8s %7.7s %s\n",
00086                 pthreadInfo->name,
00087                 (void *)pthreadInfo,
00088                 (unsigned long)pthreadInfo->lwpId,
00089                 pthreadInfo->osiPriority, priority,
00090                 pthreadInfo->isSuspended ?  "SUSPEND" :  "OK",
00091                 policies, cpuspec);
00092     }
00093 }
00094
00100 static void mcoreThreadInfo(epicsThreadId id)
00101 {
00102     mcoreThreadShowPrint(id, showLevel);
00103 }
00104
00111 static void mcoreThreadInfoOne(epicsThreadId id)
00112 {
00113     intptr_t u = id->lwpId;
00114     if (id == showThread || (epicsThreadId) u == showThread) {
00115         mcoreThreadInfo(id);
00116     }
00117 }
00118
00122 void mcoreThreadShow(epicsThreadId thread, unsigned int level)
00123 {
00124     if (!thread) {
00125         mcoreThreadShowPrint(0, level);
00126         return;
00127     }
00128     showThread = thread;
00129     showLevel = level;
00130     epicsThreadMap(mcoreThreadInfoOne);
00131 }
00132
00136 void mcoreThreadShowAll(unsigned int level)
00137 {
00138     showLevel = level;
00139     mcoreThreadShowPrint(0, level);
00140     epicsThreadMap(mcoreThreadInfo);
00141 }
00142
00143 static void once(void *arg)
00144 {
00145     cpuDigits = (int) log10(NO_OF_CPUS-1) + 1;
00146     if (!buffer)  buffer  = (char *) calloc(cpuDigits+2, sizeof(char));
00147     if (!cpuspec) cpuspec = (char *) calloc(NO_OF_CPUS + 2, sizeof(char));
00148     printf("MCoreUtils version " VERSION "\n");
00149 }
```

```
00150
00154 void mcoreThreadShowInit(void)
00155 {
00156     static epicsThreadOnceId onceFlag = EPICS_THREAD_ONCE_INIT;
00157     epicsThreadOnce(&onceFlag, once, NULL);
00158 }
00159
```

5.11 utils.c File Reference

Utility functions for MCoreUtils.

```
#include <stdlib.h>
#include <stdio.h>
#include <sched.h>
#include <string.h>
#include <errlog.h>
#include <shareLib.h>
#include "utils.h"
```

Functions

- void strToCpuset (cpu_set_t *cpuset, const char *spec)

  Convert a cpuset string specification (e.g. "0,2-3") to a cpuset.
- void cpusetToStr (char *set, size_t len, const cpu_set_t *cpuset)

  Convert a cpuset into its string specification (e.g. "0,2-3").
- const char * policyToStr (const int policy)

  Convert scheduling policy to string.
- int strToPolicy (const char *string)

  Convert string policy specification to policy.

Variables

- epicsShareDef int cpuDigits

5.11.1 Detailed Description

Utility functions for MCoreUtils.

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file utils.c.

5.11.2 Function Documentation

5.11.2.1 cpusetToStr()

```
void cpusetToStr (
            char * set,
            size_t len,
            const cpu_set_t * cpuset )
```

Convert a cpuset into its string specification (e.g. "0,2-3").

Parameters

set	output buffer to write into
len	length of `set`
cpuset	cpuset to convert

Definition at line 63 of file utils.c.

5.11.2.2 policyToStr()

```
const char * policyToStr (
            const int policy )
```

Convert scheduling policy to string.

Parameters

policy	policy to convert

Returns

string representation

Definition at line 101 of file utils.c.

5.11.2.3 strToCpuset()

```
void strToCpuset (
            cpu_set_t * cpuset,
            const char * spec )
```

Convert a cpuset string specification (e.g. "0,2-3") to a cpuset.

Parameters

cpuset	cpuset to write into
spec	specification string

Definition at line 33 of file utils.c.

5.11.2.4 strToPolicy()

```
int strToPolicy (
            const char * string )
```

Convert string policy specification to policy.

Parameters

string	string policy specification

Returns

policy value, or -1 on error

Definition at line 129 of file utils.c.

5.11.3 Variable Documentation

5.11.3.1 cpuDigits

```
epicsShareDef int cpuDigits
```

Definition at line 25 of file utils.c.

5.12 utils.c

Go to the documentation of this file.

```
00001 /*************************************************/
00012 #include <stdlib.h>
00013 #include <stdio.h>
00014 #include <sched.h>
00015 #include <string.h>
00016
00017 #include <errlog.h>
00018 #include <shareLib.h>
00019
00021 #define epicsExportSharedSymbols
00023 #include "utils.h"
```

```
00024
00025 epicsShareDef int cpuDigits;
00026
00033 void strToCpuset(cpu_set_t *cpuset, const char *spec)
00034 {
00035     char *buff = strdup(spec);
00036     char *tok, *save = NULL;
00037
00038     CPU_ZERO(cpuset);
00039
00040     tok = strtok_r(buff, ",", &save);
00041     while (tok) {
00042         int i;
00043         int from, to;
00044         from = to = atoi(tok);
00045         char *sep = strstr(tok, "-");
00046         if (sep) {
00047             to = atoi(sep+1);
00048         }
00049         for (i = from; i <= to; i++) {
00050             CPU_SET(i, cpuset);
00051         }
00052         tok = strtok_r(NULL, ",", &save);
00053     }
00054 }
00055
00063 void cpusetToStr(char *set, size_t len, const cpu_set_t *cpuset)
00064 {
00065     int cpu = 0;
00066     int l;
00067     char buf[cpuDigits*2+3];
00068
00069     if (!set || !len) return;
00070     set[0] = '\0';
00071     while (cpu < NO_OF_CPUS) {
00072         int from, to;
00073         while (!CPU_ISSET(cpu, cpuset) && cpu < NO_OF_CPUS) {
00074             cpu++;
00075         }
00076         if (cpu >= NO_OF_CPUS) {
00077             break;
00078         }
00079         from = to = cpu++;
00080         while (CPU_ISSET(cpu, cpuset) && cpu < NO_OF_CPUS) {
00081             to = cpu++;
00082         }
00083         if (from == to) {
00084             sprintf(buf, "%d,", from);
00085         } else {
00086             sprintf(buf, "%d-%d,", from, to);
00087         }
00088         strncat(set, buf, (len - 1 - strlen(set)));
00089     }
00090     if ((l = strlen(set))) {
00091         set[l-1] = '\0';
00092     }
00093 }
00094
00101 const char *policyToStr(const int policy)
00102 {
00103     switch (policy) {
00104     case SCHED_OTHER:
00105         return "OTHER";
00106     case SCHED_FIFO:
00107         return "FIFO";
00108     case SCHED_RR:
00109         return "RR";
00110 #ifdef SCHED_BATCH
00111     case SCHED_BATCH:
00112         return "BATCH";
00113 #endif /* SCHED_BATCH */
00114 #ifdef SCHED_IDLE
00115     case SCHED_IDLE:
00116         return "IDLE";
00117 #endif /* SCHED_IDLE */
00118     default:
00119         return "?";
00120     }
00121 }
00122
00129 int strToPolicy(const char *string)
00130 {
00131     int policy = -1;
00132     if (string == strcasestr(string, "SCHED_")) {
00133         string += 6;
00134     }
00135     if (0 == strncasecmp(string, "OTHER", 1)) {
```

```
00136         policy = SCHED_OTHER;
00137     } else if (0 == strncasecmp(string, "FIFO", 1)) {
00138         policy = SCHED_FIFO;
00139     } else if (0 == strncasecmp(string, "RR", 1)) {
00140         policy = SCHED_RR;
00141     }
00142 #ifdef SCHED_BATCH
00143     else if (0 == strncasecmp(string, "BATCH", 1)) {
00144         policy = SCHED_BATCH;
00145     }
00146 #endif /* SCHED_BATCH */
00147 #ifdef SCHED_IDLE
00148     else if (0 == strncasecmp(string, "IDLE", 1)) {
00149         policy = SCHED_IDLE;
00150     }
00151 #endif /* SCHED_IDLE */
00152     else {
00153         errlogPrintf("Invalid policy \"%s\"\n", string);
00154         return -1;
00155     }
00156     return policy;
00157 }
```

5.13 utils.h File Reference

Header file for utils.c.

```
#include <sched.h>
#include <unistd.h>
#include <errlog.h>
#include <shareLib.h>
```

Macros

- #define NO_OF_CPUS sysconf(_SC_NPROCESSORS_CONF)
- #define checkStatus(status, message)

Functions

- void strToCpuset (cpu_set_t *cpuset, const char *spec)

  Convert a cpuset string specification (e.g. "0,2-3") to a cpuset.
- void cpusetToStr (char *set, size_t len, const cpu_set_t *cpuset)

  Convert a cpuset into its string specification (e.g. "0,2-3").
- const char * policyToStr (const int policy)

  Convert scheduling policy to string.
- int strToPolicy (const char *string)

  Convert string policy specification to policy.

Variables

- epicsShareExtern int cpuDigits

  Number of digits needed for a single CPU spec.

5.13.1 Detailed Description

Header file for utils.c.

Author

Ralph Lange Ralph.Lange@gmx.de

Copyright

Definition in file utils.h.

5.13.2 Macro Definition Documentation

5.13.2.1 checkStatus

```
#define checkStatus(
              status,
              message )
```

Value:

```
if((status))  {\
    errlogPrintf("%s error %s\n", (message), strerror((status))); \
}
```

Definition at line 24 of file utils.h.

5.13.2.2 NO_OF_CPUS

```
#define NO_OF_CPUS sysconf(_SC_NPROCESSORS_CONF)
```

Definition at line 22 of file utils.h.

5.13.3 Function Documentation

5.13.3.1 cpusetToStr()

```
void cpusetToStr (
             char * set,
             size_t len,
             const cpu_set_t * cpuset )
```

Convert a cpuset into its string specification (e.g. "0,2-3").

Parameters

set	output buffer to write into
len	length of `set`
cpuset	cpuset to convert

Definition at line 63 of file utils.c.

5.13.3.2 policyToStr()

```
const char * policyToStr (
            const int policy )
```

Convert scheduling policy to string.

Parameters

policy	policy to convert

Returns

string representation

Definition at line 101 of file utils.c.

5.13.3.3 strToCpuset()

```
void strToCpuset (
            cpu_set_t * cpuset,
            const char * spec )
```

Convert a cpuset string specification (e.g. "0,2-3") to a cpuset.

Parameters

cpuset	cpuset to write into
spec	specification string

Definition at line 33 of file utils.c.

5.13.3.4 strToPolicy()

```
int strToPolicy (
```

```
const char * string )
```

Convert string policy specification to policy.

Parameters

string	string policy specification

Returns

policy value, or -1 on error

Definition at line 129 of file utils.c.

5.13.4 Variable Documentation

5.13.4.1 cpuDigits

```
epicsShareExtern int cpuDigits
```

Number of digits needed for a single CPU spec.

Set in mcoreThreadShowInit().

Definition at line 38 of file utils.h.

5.14 utils.h

Go to the documentation of this file.

```
00001 /*************************************************/
00012 #ifndef UTILS_H
00013 #define UTILS_H
00014
00015 #include <sched.h>
00016 #include <unistd.h>
00017
00018 #include <errlog.h>
00019 #include <shareLib.h>
00020
00021 // TODO: Use libCom call when get-cpus branch is merged
00022 #define NO_OF_CPUS sysconf(_SC_NPROCESSORS_CONF)
00023
00024 #define checkStatus(status,message) \
00025 if((status))  {\
00026 errlogPrintf("%s error %s\n", (message), strerror((status))); \
00027 }
00028
00029 #ifdef __cplusplus
00030 extern "C" {
00031 #endif
00032
00038 epicsShareExtern int cpuDigits;
00039
00040 void strToCpuset(cpu_set_t *cpuset, const char *spec);
00041 void cpusetToStr(char *set, size_t len, const cpu_set_t *cpuset);
00042 const char *policyToStr(const int policy);
00043 int strToPolicy(const char *string);
00044
00045 #ifdef __cplusplus
00046 }
00047 #endif
00048
00049 #endif // UTILS_H
```

Index